



초보(처음 내딛는 걸음)자의 코딩 기초



R 데이터 리터리시 - 작은 차이를 이해하는 힘, R로 시작하기



CLI

</> 코딩 입문자



데이터 리터리시



문법 감각

특수문자, 순서, 대소문자
차이점 이해하기



자료형

숫자형, 문자형, 논리형
시간형 구분



연산자

할당 연산자, 산술 연산
비교 연산



문법 감각



자료형



연산자



RStudio 팁

🎯 강의 목표

R 프로그래밍의 핵심 개념을 단계적으로 익혀 데이터 리터러시를 완성합니다



문법 민감성 이해

1

R의 **특수문자**, **순서**, **대소문자** 등 문법 민감성을 정확히 파악하고, **>=**와 **=>**의 차이점을 이해합니다.

- ✓ 특수문자 순서 확인하기
- ✓ 대소문자 구분 방법 익히기



자료형 구분

2

숫자형, **문자형**, **논리형**, **시간형** 4가지 자료형을 구분하고 **class()** 함수로 확인합니다.

- ✓ 각 자료형의 특성 파악
- ✓ 자료형 확인 방법 익히기



연산자 활용

3

할당 연산자(**=**, **<-**)와 **=**의 사용 차이를 이해하고, 산술 연산과 비교 연산을 정확히 수행합니다.

- ✓ 할당 연산자 방향 이해
- ✓ 산술/비교 연산 구분



RStudio 활용

4

자동완성, **스크립트 작성**, **그래프 저장** 등 RStudio의 실용적인 기능을 활용합니다.

- ✓ 자동완성 기능 활용
- ✓ 그래프 저장 방법 익히기

? 왜 코딩이 어렵게 느껴질까?

코딩의 어려움을 이해하고 긍정적인 학습 태도를 갖추습니다

⚠ 기호 하나, 순서 하나가 결과를 바꿉니다

사람 눈에는 같은 말로 보이지만, 컴퓨터에게는 **전혀 다른 의미**가 될 수 있습니다. `3 >= 1`은 올바르지만 `3 => 1`은 에러를 발생시킵니다.

```
# 올바른 코드
> 3 >= 1
[1] TRUE

# 잘못된 코드
> 3 => 1
Error: '=>' is disabled
```

👁 컴퓨터와 사람의 사고 방식이 다릅니다

컴퓨터는 **엄격한 규칙**을 따르며, 사람의 직관과 다르게 작동합니다. 작은 차이에도 민감하게 반응하여 에러를 발생시킵니다.

💡 해결 방법

확인하고 이해한다는 습관을 들입니다. 궁금한 것은 직접 입력해서 결과를 확인하고, 작은 시행착오를 통해 배우는 것이 중요합니다.

🧠 긍정적인 학습 태도

- 1 **외우기보다 확인하기**
직접 입력해서 결과를 확인하는 습관을 들입니다
- 2 **작은 실험 반복하기**
작은 코드 블록으로 실험하며 익숙해집니다
- 3 **에러를 두려워하지 않기**
에러 메시지는 학습의 길잡이가 됩니다
- 4 **기록 습관 만들기**
배운 내용을 정리하여 복습합니다

</> 특수문자 순서 비교

코드에서 `>=`와 `=>`의 차이점을 이해합니다

✅ 올바른 사용 (`>=`)

정상

```
1 # 3이 1보다 크거나 같다
2 3 >= 1
3 # 결과: [1] TRUE
4
5 # 3이 1보다 크거나 같다
6 3 >= 1
7 # 결과: [1] TRUE
```

✅ 부등호(`>`, `<`)와 등호(`=`)의 순서가 올바릅니다

❌ 잘못된 사용 (`=>`)

에러

```
1 # 3이 1보다 같거나 크다
2 3 => 1
3 # Error: '=>' is disabled
4
5 # 3이 1보다 같거나 크다
6 3 => 1
7 # Error: unexpected '='
```

⚠️ 등호(`=`)와 부등호(`>`)의 순서가 바뀌었습니다

💡 핵심 포인트

→ 순서가 중요합니다: 부등호(`>`, `<`)와 등호(`=`)의 순서를 바꾸면 완전히 다른 의미가 됩니다. `>=`는 "크거나 같다"를 의미하지만, `=>`는 R에서 인식하지 못하는 연산자입니다.

올바른 순서

`>=` : 부등호 먼저, 등호 나중
`3 >= 1` → TRUE

잘못된 순서

`=>` : 등호 먼저, 부등호 나중
`3 => 1` → Error

A 대소문자 구분

R에서 "a"와 "A"는 완전히 다른 문자입니다

✓ 소문자 vs 소문자

TRUE

```
1 # 소문자 a와 소문자 a 비교
2 "a" == "a"
3 # 결과: [1] TRUE
4
5 # 같은 문자끼리는 같다고 인식
6 "hello" == "hello"
7 # 결과: [1] TRUE
```

✓ 대소문자가 일치하면 TRUE

✗ 소문자 vs 대문자

FALSE

```
1 # 소문자 a와 대문자 A 비교
2 "a" == "A"
3 # 결과: [1] FALSE
4
5 # 다른 문자로 인식
6 "Hello" == "hello"
7 # 결과: [1] FALSE
```

⚠ 대소문자가 다르면 FALSE

💡 핵심 포인트

→ R은 대소문자를 구분합니다: 변수명, 문자열, 함수명 모두 대소문자를 엄격하게 구분합니다. "a"와 "A"는 완전히 다른 문자로 인식됩니다.

대소문자 일치

"a" == "a" → TRUE

같은 문자끼리는 같다고 인식

대소문자 불일치

"a" == "A" → FALSE

다른 문자로 인식

⚠ 주의: 변수명도 대소문자를 구분합니다. data, Data, DATA는 모두 다른 변수입니다.



확인하고 이해하기

직접 실험하며 배우는 방법론 - R의 답을 믿고, 이유는 차차 이해합니다



직접 확인하는 습관

궁금한 것은 **직접 물어보고** 확인하는 것이 중요합니다. R은 질문에 대한 답을 바로 보여줍니다.

```
> "a" == "A"
[1] FALSE # 소문자 a와 대문자 A는 다름
> "a" == "a"
[1] TRUE # 같은 문자는 같음
```

💡 실험 → 관찰 → 기록(주석/노트)의 과정을 반복하세요



학습 프로세스

- 1 실험: 직접 코드를 입력해보기
- 2 관찰: 결과를 주의 깊게 관찰하기
- 3 기록: 주석이나 노트에 정리하기



이해하는 방식

R의 답을 **믿고**, 이유는 **차차 이해**합니다. 처음부터 완벽히 이해하려 하지 마세요.

```
> class("hello")
[1] "character" # 문자형 확인
> class(123)
[1] "numeric" # 숫자형 확인
```

✅ **class()** 함수로 자료형을 확인하는 습관을 들이세요



학습 팁

확인하고 이해한다는 것: 외우기보다 직접 입력해서 확인하는 것이 이해하는 가장 빠른 방법입니다.

“ R의 답을 믿고, 이유는 차차 이해합니다 ”

</> 코딩 스타일 가이드

R 프로그래밍에서 **소문자 사용 원칙**과 **변수명 규칙**을 체크리스트로 확인합니다

A 소문자 사용 원칙

코드는 소문자로만 작성

- ✓ **변수명** 은 소문자로 시작합니다

```
my_variable # ✓ 올바른
MyVariable # ✗ 권장하지 않음
```

- ✓ **함수명** 도 소문자로 작성

```
mean(), sum(), plot() # ✓ 올바른
```

- ✓ 대문자는 **코딩에 익숙해진 후** 사용

☰ 변수명 규칙

단어 연결 규칙

- ✓ **언더스코어(_)** 로 단어 연결

```
jump_and_spring # ✓ 올바른
jump-and-spring # ✗ 에러
```

- ✓ **공백** 사용 금지

```
my variable # ✗ 에러
```

- ✓ **숫자** 로 시작 불가

```
1a # ✗ 에러
```

💡 핵심 팁

코드 가독성을 위해 **의미 있는 이름**을 사용하세요. **my_data**보다는 **student_scores**처럼 **목적이 명확한 이름**을 선택하세요.

</> 변수명 작성 규칙

R 프로그래밍에서 올바른 변수명을 작성하는 규칙과 가이드를 확인합니다

A 기본 규칙

변수명 작성의 기본 원칙

- ✓ 알파벳으로 시작 해야 합니다

```
my_variable # ✓ 올바른
1a # ✗ 에러 (숫자로 시작)
```

- ✓ 소문자로 시작 하는 것을 권장

```
student_name # ✓ 권장
StudentName # ✗ 권장하지 않음
```

- ✓ 의미 있는 이름 사용하기

작성 규칙

단어 연결과 특수문자

- ✓ 언더스코어(_) 로 단어 연결

```
jump_and_spring # ✓ 올바른
jump and spring # ✗ 에러 (공백)
```

- ✓ 하이픈(-) 사용 금지

```
jump-and-spring # ✗ 에러
```

- ✓ 특수문자 사용 주의

💡 핵심 팁

변수명은 의미가 명확해야 합니다. a, b, c보다는 student_scores, total_sum처럼 목적이 드러나는 이름을 사용하세요.

</> 언더스코어(_) 사용법

올바른 변수명과 잘못된 변수명을 비교하여 학습합니다



올바른 사용

언더스코어(_)로 단어 연결

```
# 단어와 단어를 연결할 때
jump_and_spring # ✓ 올바른
my_variable # ✓ 올바른
data_frame # ✓ 올바른
```

✓ 언더스코어(_)는 단어 연결에 적합합니다

- 단어와 단어 사이에 사용
- 가독성이 좋음
- R에서 인식 가능



잘못된 사용

하이픈(-)이나 공백 사용

```
# 단어와 단어를 연결할 때
jump-and-spring # ✗ 에러
my variable # ✗ 에러
JumpAndSpring # ✗ 권장하지 않음
```

! 하이픈(-)은 연산자로 인식됩니다

- 하이픈(-)은 뺄셈 연산자
- 공백은 변수명에 사용 불가
- 대문자는 초반 권장 안함

✓ 올바른 예시

student_scores, data_frame, my_variable

✗ 잘못된 예시

student-scores, data frame, MyVariable

→ 할당연산자 -> (오른쪽 화살표)

왼쪽 값을 오른쪽 변수에 할당하는 방법을 학습합니다

→ 기본 개념



```
1 # 숫자 1,2,3,4,5,6을 A 변수에 할당
2 c(1,2,3,4,5,6) ->A
3
4 # A 변수 확인
5 A
6 # [1] 1 2 3 4 5 6
```

💡 핵심 포인트

→ 화살표 방향이 중요합니다:>는 왼쪽에서 오른쪽으로 값을 전달합니다. 왼쪽에 있는 값을 오른쪽 변수에 할당하는 역할을 합니다.

📊 실제 예시

실습

```
1 # 3+4의 결과를 a에 할당
2 3+4->a
3
4 # a 확인
5 a
6 # [1] 7
```

📌 할당 연산자의 특징

연산 결과를 변수에 저장할 수 있고, 변수를 변수에 넣을 수도 있습니다.

값 (Value)



왼쪽

할당연산자



->

변수 (Variable)



오른쪽

← 할당연산자 <- 설명

왼쪽 화살표(<-)를 사용하여 값을 변수에 할당하는 방법

✓ 기본 사용법

정상

```
1 # 오른쪽 값을 왼쪽 변수에 할당
2 1 + 2<-b
3 # 결과: b에 3이 저장됨
4
5 # 벡터 할당
6 c(10, 9, 8, 7) <-B
7 # B: 10 9 8 7
```

✓ 왼쪽 변수에 오른쪽 값을 할당합니다

📊 계산 결과 저장

정상

```
1 # 연산 결과를 변수에 저장
2 3 + 4<-a
3 # a에 7이 저장됨
4
5 # 변수 간 연산
6 a + b<-c
7 # c에 10이 저장됨
```

i 연산 결과를 변수에 저장할 수 있습니다

💡 핵심 포인트

값: 3 + 4 ← 변수: a = a = 7

→ 화살표 방향이 중요합니다: <-는 오른쪽의 값을 왼쪽 변수에 넣습니다. **value <- variable** 형태로 사용됩니다.

= 이퀄(=) 연산자

= 사용법과 주의점을 코드로 설명합니다

✅ 올바른 사용

정상

```
1 # 변수에 값 할당
2 d=3+4
3 # d는 7이 됨
4
5 d
6 # [1] 7
```

✅ 오른쪽 계산 결과를 왼쪽 변수에 할당

❌ 잘못된 사용

에러

```
1 # 왼쪽에 계산식, 오른쪽에 변수
2 3+4=d
3 # Error: target of assignment...
4
5 3+4=e
6 # Error: object 'e' not found
```

⚠️ =는 오른쪽 결과를 왼쪽에 담는 형식만 허용

💡 핵심 포인트

→ = 연산자의 원칙: =는 오른쪽의 계산 결과를 왼쪽 변수에 할당합니다. 순서가 바뀌면 에러가 발생합니다.

올바른 형식

변수 = 값

d = 3 + 4 → 정상

잘못된 형식

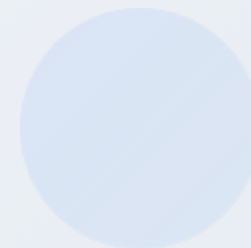
값 = 변수

3 + 4 = d → 에러



자료형 개념 소개

컴퓨터가 인식하는 데이터의 4가지 종류를 알아보시다



숫자형 (Numeric)

숫자를 의미하는 자료형입니다. 덧셈, 뺄셈, 곱셈, 나눗셈 등 산술연산이 가능합니다.

```
3 + 4 -> a
```



문자형 (Character)

글자나 문자를 의미하는 자료형입니다. 따옴표로 감싸서 만듭니다.

```
"3 + 4" -> b
```



논리형 (Logical)

TRUE 또는 FALSE 결과를 담은 자료형입니다. 비교 연산의 결과로 생성됩니다.

```
3 > 2 -> c
```



시간형 (POSIXct)

날짜와 시간을 다루는 자료형입니다. 복잡한 진수를 가지고 있습니다.

```
Sys.time() -> d
```

숫자형(Numeric) - 개념과 예시

R에서 숫자형 데이터의 특징과 활용 방법을 이해합니다

숫자형 개념

정의

숫자형은 이름 그대로 **숫자**를 의미하는 자료형입니다. R에서 숫자형은 덧셈, 뺄셈, 곱셈, 나눗셈 등의 **산술연산**이 가능한 가장 기본적인 데이터 타입입니다.

정수

1, 2, 3, -5, 0

실수

3.14, 2.5, -1.5

💡 **특징:** 숫자형은 **numeric**으로 분류되며, `class()` 함수로 확인할 수 있습니다.

</> 실제 예시

```
1 # 3 + 4의 결과를 a에 저장
2 3+4->a
3
4 # a의 값 확인
5 >a
6 [1] 7
7
8 # 자료형 확인
9 >class(a)
10 [1] "numeric"
```

결과:

a = 7 (숫자형)

A 문자형(Character) 자료형

따옴표로 감싸서 만드는 문자 데이터, "3 + 4"는 계산이 아닌 문자열

문자형 개념

문자형은 숫자나 글자를 ""로 감싸서 만듭니다. 따옴표 안에 있는 내용은 모두 **문자열**로 인식됩니다.

```
1 # "3 + 4"를 변수 b에 저장
2 "3 + 4" -> b
3
4 # b의 값 확인
5 > b
6 [1] "3 + 4"
7
8 # 자료형 확인
9 > class(b)
10 [1] "character"
```

핵심 포인트

따옴표 안에 있으면 **모두 문자**로 인식됩니다. "3 + 4"는 계산식이 아닌 **문자열**입니다.

</> 문자형 특징

문자형은 **산술연산이 불가능**합니다. 숫자처럼 보이지만 실제로는 텍스트입니다.

"Hello"

문자열

"123"

문자열

"3+4"

문자열

"TRUE"

문자열

⚠ 주의: 문자형은 +, -, *, / 등의 산술연산을 할 수 없습니다.

문자형

"3" + "4" = Error

숫자형

3 + 4 = 7

</> 문자형 산술연산 불가

문자형 데이터는 산술연산이 불가능합니다

✖ 문자형 연산 시도 (에러)

에러

```
1 # 문자형 변수 생성
2 "3 + 4" -> b
3 b
4 # [1] "3 + 4"
5
6 # 문자형 연산 시도
7 b + b
8 # Error: non-numeric argument to binary operator
```

⚠ 문자형은 산술연산을 할 수 없습니다

✔ 숫자형 연산 (정상)

정상

```
1 # 숫자형 변수 생성
2 3+4 -> a
3 a
4 # [1] 7
5
6 # 숫자형 연산
7 a + a
8 # [1] 14
```

✔ 숫자형은 산술연산이 가능합니다

⚠ 주의사항

문자형 데이터는 계산 대상이 아닙니다. "3 + 4"는 문자 그대로 "3 + 4"이며, 계산 결과 7이 아닙니다.

💡 핵심 포인트

➔ 자료형 확인이 중요: `class()` 함수로 자료형을 확인하고, 연산 가능 여부를 판단하세요.

숫자형
numeric
연산 가능

문자형
character
연산 불가

✓ 논리형(Logical) - TRUE/FALSE

비교 연산의 결과로 생성되는 논리값을 이해합니다

</> 논리형 생성 예시

비교 연산의 결과로 논리형이 생성됩니다

```
1 # 3이 2보다 크다
2 3>2->c
3 # 결과: [1] TRUE
4
5 # 자료형 확인
6 class(c)
7 # [1] "logical"
```

🗪 논리값 비교표

비교	결과	설명
3 > 2	TRUE	3은 2보다 크다
3 == 3	TRUE	3은 3과 같다

💡 논리형 활용

논리형은 다음 상황에서 활용됩니다:

- 필터링 데이터 조건에 맞는 행 선택
- 조건 분기 if문, while문 등 제어 구조
- 논리 연산 AND, OR, NOT 연산

⚠️ 주의사항

논리값은 대소문자를 구분합니다

TRUE와 true는 다릅니다. R에서는 대문자 TRUE/FALSE를 사용합니다.

🕒 시간형: 날짜와 시간 다루기

R에서 시간 데이터를 다루는 방법과 lubridate 패키지 활용

📄 기본 시간형 사용

```
1 # 현재 시간 확인
2 Sys.time() -> d
3 # 결과: [1] "2026-05-17 21:15:42 KST"
4
5 # 자료형 확인
6 class(d)
7 # [1] "POSIXct" "POSIXt"
```

📌 시간형의 특징

- ✓ 복잡한 진수: 초, 분은 60진수, 시는 24진수 사용
- ✓ 날짜 규칙: 1월은 30일/31일, 2월은 28일/29일(윤달)
- ✓ 자료형: "POSIXct"와 "POSIXt" 두 가지 클래스

📄 lubridate 패키지 사용법

시간/날짜를 직관적이고 편리하게 다룰 수 있는 패키지

```
# 패키지 설치 및 로드
install.packages("lubridate")
library(lubridate)

# 날짜 파싱
ymd("2026-05-17")
mdy("05-17-2026")
dmy("17-05-2026")
```

💡 실용적인 팁

- ★ ymd(), mdy(), dmy(): 날짜 형식에 맞게 선택
- ★ now(): 현재 시간, today(): 오늘 날짜
- ★ with_tz(): 시간대 변환

🔍 class() 함수로 자료형 확인하기

만든 변수의 자료형을 정확히 파악하는 방법을 배웁니다

</> class() 함수 사용 예시

```
1 # 숫자형 확인
2 3+4->a
3 class(a)
4 # [1] "numeric"
5
6 # 문자형 확인
7 "3 + 4" ->b
8 class(b)
9 # [1] "character"
```

🕒 시간형 확인

```
1 Sys.time() ->d
2 class(d)
3 # [1] "POSIXct" "POSIXt"
```

💡 자료형별 특징

숫자형 (numeric)

연산 가능

class(a) → "numeric"

✓ 수학 연산 가능

문자형 (character)

텍스트

class(b) → "character"

✓ 따옴표로 감싸서 생성

논리형 (logical)

TRUE/
FALSE

class(c) → "logical"

✓ 비교 연산 결과

시간형 (POSIXct)

날짜/시간

class(d) → "POSIXct"

✓ 복잡한 시간 구조

⚠️ 중요한 팁

class() 함수는 변수의 자료형을 확인하는 가장 기본적인 방법입니다. 변수를 만들고 나면 항상 class()로 자료형을 확인하는 습관을 들이세요.

</> 숫자형 vs 문자형 비교

숫자형(Numeric)과 문자형(Character)의 차이점을 비교하여 이해합니다



숫자형 (Numeric)

수학 연산이 가능한 자료형

```
# 숫자형 예시
7 + 3 # 10 (계산 가능)
10 * 2 # 20 (계산 가능)
class(7) # "numeric"
```

✓ 산술 연산이 가능합니다

- 덧셈, 뺄셈, 곱셈, 나눗셈 가능
- 수학적 계산에 사용
- 숫자만 포함 (따옴표 없음)

✓ 숫자형 특징

```
7 + 3 = 10
10 * 2 = 20
class(7) = "numeric"
```



문자형 (Character)

텍스트를 저장하는 자료형

```
# 문자형 예시
"7" + "3" # 에러!
"Hello" * 2 # 에러!
class("7") # "character"
```

! 산술 연산이 불가능합니다

- 계산 대상이 아님
- 텍스트 처리에 사용
- 따옴표로 감싸야 함

✗ 문자형 특징

```
"7" + "3" = 에러
"Hello" * 2 = 에러
class("7") = "character"
```

</> 변수 작성 규칙

첫 글자 제약과 올바른 변수명 작성법을 알아보시다

⚠ 첫 글자 규칙

변수는 **알파벳(문자)**로 시작해야 하며, 숫자나 특수문자로 시작할 수 없습니다.

```
1 # 올바른 변수명 (문자로 시작)
2 3 + 4->a
3 a# 결과: [1] 7
4
5 # 잘못된 변수명 (숫자로 시작)
6 3 + 4->1a
7 Error: unexpected symbol in "3 + 4 -> 1a"
```

💡 중요한 점

변수명은 **알파벳 대문자, 소문자**로 시작해야 하며, 숫자나 특수문자를 첫 글자로 사용할 수 없습니다.

✓ 올바른 변수명 예시

소문자 시작

`my_var`

대문자 시작

`MyVar`

언더스코어

`my_var_1`

숫자 포함

`var1`

✗ 잘못된 변수명 예시

숫자로 시작

`1a`

특수문자

`$var`

공백 포함

`my var`

하이픈

`my-var`

i 변수명 작성 팁

→ 의미 있는 이름을 사용하세요: `student_score`보다 `final_exam_score`가 더 명확합니다.

→ 소문자로 시작하는 것이 R의 관례입니다.

! 자주 하는 실수 모음

R 프로그래밍에서 자주 발생하는 5가지 오류를 확인하고 해결 방법을 알아봅시다

> 1. 연산자 오타

>=를 =>로 잘못 입력

```
3 => 1 # 에러!  
3 >= 1 # ✓ 올바른
```

할당연산자와 비교연산자 순서 주의

- 2. 하이픈 사용

- 대신 _ 사용

```
my-data # 에러!  
my_data # ✓ 올바른
```

변수명에 하이픈 사용 금지

” 3. 따옴표 누락

문자열에 " 누락

```
3 + 4 -> a# 변수로 인식  
"3" + "4" -> b# ✓
```

문자열은 반드시 따옴표로 감싸기

A 4. 대소문자 불일치

data vs Data

```
data # 소문자  
Data # 대문자 (다른 변수)
```

R은 대소문자를 구분함

= 5. 할당 방향 오류

= 사용 시 방향 주의

```
3 + 4 = d# 에러!  
3 + 4 -> d# ✓
```

할당연산자 방향 확인

Q 6. 변수 미선언

사용 전 변수 선언 확인

```
print(x) # x 없음  
x <- 10# 먼저 선언
```

변수 선언 후 사용

💡 해결 방법

에러 메시지를 천천히 읽고 키워드를 검색하세요. 작은 실험으로 원인을 분리하고, 기록 습관을 들이면 해결이 쉬워집니다.

❤ 에러를 두려워하지 않기

💡 긍정적인 학습 태도

에러는 학습의 일부, 메시지로 길을 찾아가는 과정

🔍 재현과 분석

에러가 발생하면 **재현**을 시도하고, **최소 예제**로 원인을 분리합니다. 메시지를 천천히 읽으며 키워드로 검색합니다.

- ✓ 에러 메시지 분석하기
- ✓ 최소 예제로 테스트

🧪 작은 실험

작은 실험으로 원인을 분리하고, **직접 확인**하며 이해합니다. R의 답을 믿고, 이유는 차차 이해합니다.

- ✓ 작은 실험으로 원인 파악
- ✓ 직접 확인하는 습관

📖 기록 습관

나만의 **해결 노트**를 만들어 기록합니다. 에러 패턴과 해결 방법을 정리하면 실력이 향상됩니다.

- ✓ 해결 노트 만들기
- ✓ 패턴 정리하기

🔎 검색 활용

에러 메시지의 **키워드**로 검색하여 해결 방법을 찾습니다. 스택오버플로우, R 커뮤니티 활용

- ✓ 키워드 검색하기
- ✓ 커뮤니티 활용

🔄 반복 학습

같은 에러가 반복되면 **복습**이 필요합니다. 기본 개념을 다시 확인하며 깊이 있게 이해합니다.

- ✓ 기본 개념 복습
- ✓ 깊이 있는 이해

👥 커뮤니티

혼자 고민하지 말고 **동료**와 공유합니다. 함께 문제를 해결하며 성장합니다.

- ✓ 동료와 공유하기
- ✓ 함께 성장하기



RStudio 자동완성 팁

코딩 속도를 높이고 오타를 줄이는 **자동완성 기능** 활용법



자동완성 활용법

Tab 키로 빠른 입력

- 1 함수/변수명을 입력할 때 Tab 키를 누르면 자동완성 목록이 표시됩니다
- 2 함수 인자도 자동완성되어 파라미터명과 설명을 볼 수 있습니다
- 3 패키지 이름도 자동완성되어 라이브러리 로딩이 편리합니다

```
mean( # Tab 키를 누르면 함수 인자 자동완성
ggplot2:: # 패키지 함수 자동완성
```



자동완성 장점

왜 사용해야 할까?

- 1 코딩 속도 ↑ - 직접 입력보다 빠르게 코드 작성
- 2 오타 ↓ - 자동완성으로 실수 방지
- 3 일관성 ↑ - 정확한 함수명과 변수명 사용



팁

Ctrl + Space를 눌러도 자동완성 목록을 볼 수 있습니다

★ 핵심 활용법

1. 함수 이름 입력 중 Tab 누르기 - mean 입력 후 Tab → mean(x, trim = 0, na.rm = FALSE) 자동완성
2. 패키지::함수 형식 - ggplot2:: 입력 후 Tab → ggplot2 패키지의 모든 함수 목록
3. 객체 이름 완성 - 변수명의 앞글자만 입력하고 Tab → 저장된 변수 목록 표시

” 블록 따옴표 감싸기 팁

RStudio에서 **블록 선택 후 따옴표 감싸기** 기능을 활용하여 효율적으로 코딩하세요



블록 선택 후 따옴표 감싸기

텍스트 블록을 효율적으로 처리

- 1 코드 블록 을 드래그하여 선택

```
grey # 이 텍스트를 선택하세요
```

- 2 **Shift + "** 키를 누르면 자동으로 감싸짐

```
"grey" # 따옴표로 감싸짐
```

- 3 일반 텍스트, 메모장과 다르게 **RStudio**에서 유용

Shift + " 누르면 선택한 텍스트가 따옴표로 감싸집니다

텍스트 블록을 선택한 상태에서 따옴표 키를 누르세요



R Script 활용

콘솔보다 스크립트로 작성

- 1 **File > New File > R Script**로 새 파일 생성

```
# 스크립트 파일에서 코드 작성
```

- 2 **Command + Enter** (Mac) 또는 **Ctrl + Enter** (Windows)로 실행

```
# 선택한 코드 실행
```

- 3 콘솔창에 직접 입력하기보다 **소스 창**에서 작성

💡 핵심 팁

스크립트 파일을 만들어 코드를 작성하고, **블록 선택 + 따옴표** 기능으로 **문자열 편집**을 효율적으로 하세요.



그래프 이미지 저장 팁

완료

RStudio의 **Export 기능**을 활용하여 그래프를 이미지로 저장하는 방법



Export 기능 활용법

Plots 창에서 이미지로 저장

1

Plots 창에서 Export 버튼 클릭

RStudio 오른쪽 하단 Plots 창에서 Export 버튼을 클릭합니다

2

저장 형식 선택

PNG, JPEG, PDF, SVG 등 원하는 형식을 선택합니다

3

디렉토리와 파일명 설정

저장할 경로와 파일명을 입력하고 저장 버튼 클릭



해상도 설정

고품질 이미지를 위한 설정

Width

800-1200px 권장

Height

600-900px 권장



Export 설정 예시

파일 형식:

PNG

저장 경로:

~/Documents/R/

파일명:

plot_001.png

해상도:

300 DPI

💡 핵심 팁

고해상도 저장을 위해 **300 DPI** 이상 설정하세요. **PDF 형식**은 벡터 그래픽으로 무한 확대가 가능합니다.



종합 실습 - 오늘 배운 것 적용하기

6단계 실습 과제를 통해 R 프로그래밍 기초를 실습합니다

1 변수 저장과 확인

3+4의 결과를 변수 a에 저장하고 class()로 자료형을 확인하세요

```
3 + 4 -> a  
class(a)
```

✓ 결과: **numeric**

2 문자형 연산 확인

"3+4"를 변수 b에 저장하고 b+b를 시도하여 에러를 관찰하세요

```
"3+4" -> b  
b + b
```

✓ 예상: **에러 발생**

3 대소문자 구분

"a" == "A"의 결과를 확인하여 대소문자 구분을 이해하세요

```
"a" == "A"  
# 결과 확인
```

✓ 결과: **FALSE**

4 벡터 생성과 출력

c(1,2,3)를 vec에 저장하고 출력 결과를 확인하세요

```
c(1, 2, 3) -> vec  
vec
```

✓ 결과: **1 2 3**

5 시간형 확인

Sys.time()을 d에 저장하고 class(d)로 자료형을 확인하세요

```
Sys.time() -> d  
class(d)
```

✓ 결과: **POSIXct**

6 그래프 저장

간단한 plot()을 그리고 Export로 이미지를 저장하세요

```
plot(1:10)  
# Export -> Save as Image
```

✓ 파일: **.png**



핵심 요약

오늘 배운 **5가지 핵심 내용**을 간결하게 정리합니다



1. 문법 민감성

- ✓ **순서·기호·대소문자**를 철저히 확인하세요
- ✓ **>=** vs **=>** - 부등호와 등호의 순서가 중요

```
3>=1 # TRUE
3=>1 # Error
```



2. 자료형 구분

- ✓ **4가지 자료형**: numeric, character, logical, time
- ✓ **class()** 함수로 자료형 확인

```
class(a) # "numeric"
class(b) # "character"
```



3. 할당 연산자

- ✓ **->**: 왼쪽 값을 오른쪽 변수에
- ✓ **<-**: 오른쪽 값을 왼쪽 변수에

```
3+4->a
a<-3+4
```



4. 변수명 규칙

- ✓ **소문자**로 시작, **언더스코어**로 연결
- ✓ **my_variable** (O) vs **my-variable** (X)

```
jump_and_spring # ✓
jump-and-spring # X
```



5. 에러 처리

- ✓ 에러 메시지를 **천천히 읽기**
- ✓ **작은 실험**으로 원인 분리

💡 에러는 학습의 **길잡이**가 됩니다



마무리 및 다음 학습 안내

오늘 배운 내용을 정리하고 **다음 학습 방향**을 안내합니다



추천 교재



R for Data Science (영문)

R로 데이터 과학을 배우는 기본서

r4ds.had.co.nz



R for Data Science (한국어)

한국어 번역 버전

bookdown.org/sulgi/r4ds



연습문제 해답

실습 문제 해답 모음

rncold.github.io/r4ds-exercise-solutions



다음 학습 주제



벡터/데이터프레임



dplyr



ggplot2



lubridate



데이터 정제



시각화

★ 학습 순서 추천

1. 벡터와 데이터프레임 기초
2. dplyr 패키지로 데이터 조작
3. ggplot2로 시각화
4. lubridate로 날짜/시간 처리